



中国科学院大学

University of Chinese Academy of Sciences

研究生学位论文开题报告

报告题目 面向低时延大语言模型推理的 Megakernel
依赖感知物理分区与 PDL 跨边界执行研究

学生姓名 郝朝阳 学号 待填写

指导教师 待填写 职称 待填写

学位类别 工学硕士

学科专业 计算机科学与技术

研究方向 AI 编译器与 GPU 高性能计算

培养单位 处理器芯片全国重点实验室

填表日期 2026 年 07 月

中国科学院大学制

报告提纲

一 选题的背景及意义	1
1.1 研究背景	1
1.2 选题意义	4
二 国内外本学科领域的发展现状与趋势	5
2.1 从图级融合到编译期 kernel 划分	5
2.2 Persistent Megakernel	6
2.3 动态 task/event 与自动执行计划生成	6
2.4 CUDA Graph 与 Programmatic Dependent Launch	7
2.5 现状归纳与研究机会	7
三 课题主要研究内容、预期目标	8
3.1 问题定义与总体目标	8
3.2 主要研究内容	9
3.3 预期目标、创新点与成果	10
四 拟采用的研究方法、技术路线与实验方案	11
4.1 框架总体方法	11
4.2 计算图依赖关系建模	11
4.3 依赖感知的 PDL 跨边界重叠	12
4.4 全图边处理方案搜索	12
4.5 计划生成与运行时执行	12
4.6 实验方案	13
五 已有科研基础与所需的科研条件	13
5.1 已有科研基础	13
5.2 所需的科研条件	14
六 研究工作计划与进度安排	14

一 选题的背景及意义

1.1 研究背景

大语言模型 (Large Language Model, LLM) 已从离线批量生成, 转变为支撑对话、检索增强、代码补全与智能体等交互式服务的核心组件Yu 等 (2022); Kwon 等 (2023)。这类服务对单次响应时延高度敏感: 一次请求首先并行处理输入序列以产生首个 token (prefill), 随后依据已缓存的 Key-Value Cache (KV Cache) 逐 token 解码 (decode) Zhong 等 (2024); Agrawal 等 (2024)。与追求峰值吞吐的训练范式不同, 交互式推理的大量计算落在单次计算量很小的区间——小 batch 的 decode、短增量的 prefill, 以及智能体在工具调用前后需要快速返回的阶段Luo 等 (2026)。在这一阶段暴露出两个重要问题: 首先, 低 batch 推理处于与训练截然不同的约束下, 其主要瓶颈并非算力, 而是全局内存与片上内存之间频繁搬运带来的内存带宽成本Dao 等 (2022); Nrusimha 等 (2025); 其次, 为每层 Transformer 启动多个独立内核所引入的内核启动开销, 也可能上升为瓶颈Nrusimha 等 (2025); Vellaisamy 等 (2026)。尤其在低时延约束的场景中, 这些近乎固定的开销无法被充分的计算所摊薄, 会在端到端时延中占据不可忽视的比例。而解决这两类问题的主要做法, 是算子融合与启动开销的消除Chen 等 (2018); Niu 等 (2021); Shi 等 (2023); NVIDIA (2026a)。

具体而言, 一层 Transformer 推理通常被拆成归一化、线性变换、位置编码、Attention、激活函数和残差连接等多个 GPU 计算阶段; 各阶段若以独立 kernel 串行执行, 中间张量便需反复写回 HBM, 并在阶段边界支付启动与同步代价。FlashAttention 从 Attention 内部重排数据流以降低 HBM 访问; TVM、DNNFusion、Welder 则分别在计算图、融合计划与 tile-graph 层面压缩算子边界、扩大片上复用。这些工作共同表明: 低时延推理的性能, 取决于算子内部实现与算子之间执行边界的协同设计Dao 等 (2022); Chen 等 (2018); Niu 等 (2021); Shi 等 (2023)。

近期研究进一步将融合范围从局部算子扩展到 Transformer 层乃至完整解码主体。FlashFormer 这类 whole-model kernel 面向低 batch 推理, 把整模型前向融合为单个 kernelNrusimha 等 (2025); Hazy Research 以设备端指令解释器与 counter 同步, 把 Llama 前向收进单一 megakernel, 消除 kernel 边界带来的流水气泡Hazy Research (2025a,b); MPK 进一步把张量程序降低为 SM 级任务图, 并在 persistent megakernel 中去中心化调度Cheng 等 (2026); DeepGEMM 的 MegaMoE

则在 SM90 上把 dispatch、专家侧 GEMM、SwiGLU 与 combine 等阶段组织进同一 cooperative kernel DeepSeek-AI (2026a,b); Event Tensor 把跨任务依赖提升为可编译的事件张量抽象, 以支持动态 shape 与数据依赖调度 Jin 等 (2026)。这类执行方式统称为 Megakernel: CPU 只发起一次或少量物理 CUDA launch, 一组 persistent 或 cooperative workers 在 GPU 内部连续执行多个算子任务, 并通过 queue、event、counter 或 grid-wide synchronization 表达跨算子依赖。

图 1 对比同一条 $A \rightarrow B \rightarrow C$ 算子链的三种执行映射: 多 kernel、传统编译期融合, 以及 persistent Megakernel。前两者仍依赖物理 grid 边界排序阶段; Megakernel 则一次 launch, 把任务投放与依赖判断下沉到设备端, 从而在依赖允许时形成跨任务流水。

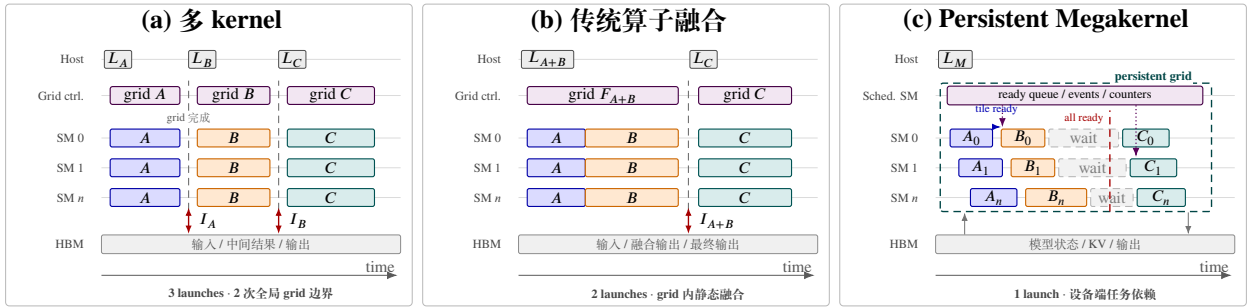


图 1 同一条 $A \rightarrow B \rightarrow C$ 算子链的三种执行映射。(a) 多 kernel 以物理 grid completion 排序阶段, 并在边界处物化中间结果; (b) 传统融合消去 $A-B$ 物理边界, 但 worker 仍执行静态融合程序, 且 C 前仍保留物理边界; (c) persistent Megakernel 仅 launch 一次, 由设备端调度维护 queue/event/counter, worker 拉取 tile 任务: tile-ready 允许 B_0 与 A 尾部重叠, 而 C 前的 all-ready 则表现为内部 readiness gate。Megakernel 中间结果是否落 HBM 取决于具体实现, 图中不作约定。

然而, 将多个算子放入同一个 Megakernel, 并不会消除算子之间的数据依赖。原先由物理 kernel 边界隐式保证的算子间的同步关系, 在融合之后仍须在设备端显式维护。以 Llama 解码阶段的计算图为例。在从 (QKV/RoPE) 到部分注意力 (PartialAttention) 的计算图连边上, 注意力计算只需等待对应注意力头的 QKV 就绪, 即可开始读取该注意力头的 KV cache; 因此, 部分注意力头可以与尚未完成的其他注意力头的前驱计算重叠执行。相比之下, 在从部分注意力到输出投影与残差连接 (O-Proj/Residual) 的计算图连边上, 每一个输出投影都依赖完整的注意力输出张量; 因此, 只有全部 query head 对应的注意力结果都已产生后, 后继的主体计算才能开始。这两种不同依赖形态决定了相邻算子边应映

射到何种物理执行方式。对于前者，将生产者与消费者保留在同一 grid 内，并通过设备端事件同步，通常能够获得有效的计算重叠。然而对于后者，过早地在同一 grid 内启动消费者往往无法形成有用重叠，却仍需承担设备端等待与跨阶段交接的开销；此时，用物理 grid 完成语义分隔前后阶段，反而可能更有利。因此，减少一次 kernel launch 与减少一次同一 grid 内的软件同步，构成方向相反的两项开销。

在这类 Megakernel 中，算子阶段还会被进一步划分成可并行调度的 task：PartialAttention 按注意力头分组，输出投影按输出行块划分。以 PartialAttention→O_ProjResidual 为例，8 个 PartialAttention task 共同产生 32 个 query head 的结果，128 个输出投影 task 却都要等这 32 个结果全部就绪后才能开始依赖计算。然而在执行时各 SM O_ProjResidual task 会在全量就绪之前就被调度并进入等待。从依赖分析的角度来看，这种提前调度也无法带来注意力计算的有效重叠，却引入了相比于一次物理 grid 的 SM 退出同步再调度更大的开销，其区别如图 2 所示。

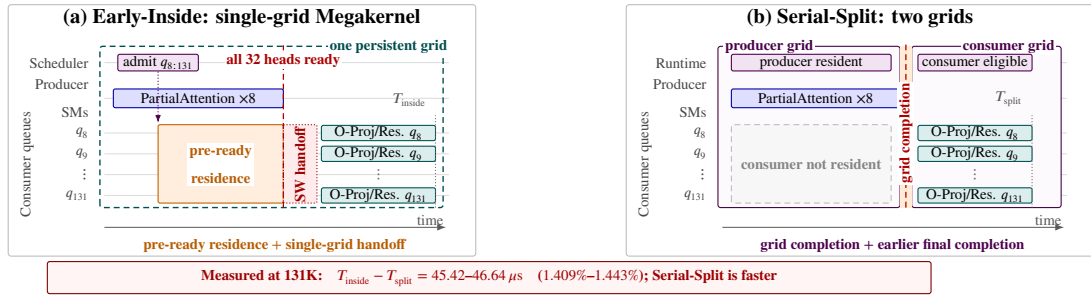


图 2 Hazy PartialAttention→O_ProjResidual 的 SM-时间执行结构。Early-Inside 中，queues 8–131 在 32 个 query heads 全量就绪前进入 consumer 路径，形成 pre-ready residence interval；Serial-Split 由 producer grid completion 统一放行 consumer grid。两种方案保持 task code、queue mapping、task/noop slots 和 instruction bytes 一致。橙色区间对应后续 admission-gated 对照与设备端时间戳所分解的关键阶段。

即使 Serial-Split 在该边上更快，生产者与消费者之间仍保留了真实的 grid 边界：后继 kernel 在依赖满足前难以尽早占用 SM，边界两侧也难以重叠利用内存带宽。Hopper 之后的架构引入的 Programmatic Dependent Launch (PDL) 正针对这一残余交接：同一 stream 上，后继 grid 可在生产者发出 programmatic completion 后提前获得调度资格，先完成地址准备、descriptor 配置和与生产者输出无关的权重预取，再在首次读取依赖数据前完成同步 NVIDIA (2026d,b)。

综上所述，低时延推理既需要跨算子融合以削减启动与访存成本，也必须

正视融合后依赖仍在：不同算子边的就绪粒度与可重叠工作量不同，因而同一 Megakernel 内不宜对所有边套用同一种同步与投放策略；物理拆分能消除部分边的无效驻留，却又留下可被 PDL 进一步压缩的交接成本。因此，如何通过依赖分析，为计算图中的各条边选择合适的执行方式，能在推理场景下得到更高的性能，而如何构建完整的推理框架为本论文关注和优化的重点。据此本论文构建一套**依赖感知的 Megakernel 推理融合框架与系统**：面向任意计算图，通过依赖分析与任务拆解生成可调度的 Megakernel 任务图；再按边判定应采用同一 grid 内的细粒度事件、内部 ready 队列的执行方式，或跨 grid 完成语义，并在选定物理边界上配置 PDL 触发与消费者独立前缀并结合数据规模优化。

1.2 选题意义

首先，本课题推进了低时延推理执行模型的认识边界。现有工作多以扩大融合范围、消除 host 侧 launch 为目标；Megakernel 把依赖判断下沉到设备端后，真正决定临界路径的，往往是算子边的就绪粒度、任务投放时机与物理执行粒度是否匹配。将同一逻辑计算图中的相邻边纳入统一分析，可以解释为何有的边应保留细粒度流水，有的边反而适合用物理 grid 完成语义隔离，从而把优化关键推进到“依赖应由哪一级机制实现”，并建立这类问题的理论模型。

其次，本课题为任意计算图上的 Megakernel 构建提供了系统化路径。面向推理场景，仅有局部手工拆边或单一 persistent grid 并不足以覆盖多样依赖形态；需要一套从依赖分析、任务拆解到执行计划生成的完整框架：先形成可调度的任务图，再按边选择同一 grid 内的事件同步、内部任务调度器，或跨 grid 完成语义，并在选定物理边界上配置 PDL 触发与消费者独立前缀。该路径使图级融合、设备端调度与硬件依赖执行落在同一决策空间，便于形成可复现、可验证的运行方案。

再次，本课题在 Megakernel 框架内把 grid 边界上的 PDL 与依赖分析结合起来，进一步整合调度时机与调度方式。依赖分析不仅回答一条边应保留在同一 grid 内，还是映射为跨 grid 完成语义，还进一步约束后继何时具备调度资格、以何种机制放行：同一 grid 内可由事件或就绪门控控制投放；跨 grid 时，除串行等待生产者完成外，还可通过 PDL 在 programmatic completion 后提前获得调度资格，并先执行合法独立前缀 [NVIDIA \(2026d,b\)](#)。由此，边界选择、触发时机与

交接方式进入同一探索空间，使“何时调度”和“怎样调度”成为依赖感知执行计划的一体两面。

最后，本课题具有直接的服务与工程价值。交互式 LLM 推理对单步时延敏感，局部临界路径上数十微秒的差异会沿 token 序列累积为可感知的 TPOT 与响应延迟。将上述分析固化为可嵌入编译器与运行时的完整系统，有望为低 batch decode、短增量 prefill 及智能体快速返回等场景提供可落地的时延收益。

二 国内外本学科领域的发展现状与趋势

2.1 从图级融合到编译期 kernel 划分

GPU 算子融合的优化单位经历了从算子节点到 tile 与更细计算片段的持续细化。TVM 在张量表达和计算图执行算子融合与跨算子优化，DNNFusion 根据算子关系和数据流属性生成融合计划，Welder 进一步以 tile graph 描述生产者与消费者之间的数据复用，从而联合调度算子内与算子间访存Chen 等 (2018); Niu 等 (2021); Shi 等 (2023)。FlashAttention 则从算法层重排 Attention 的分块与归约，使中间 attention matrix 保持在片上存储层级，展示了数据流重构与 kernel 边界设计的协同收益Dao 等 (2022); Dao (2024); Shah 等 (2024)。

直观地说，这一阶段要回答的是：一段逻辑计算应被编译成几个 GPU kernel、每个 kernel 覆盖哪些计算片段。Korch 把高层算子拆成更细的代数原语，再用优化方法决定如何把这些原语切分、拼装为最终 kernel 集合，从而允许在原始算子边界之内切开、也允许跨越边界重组Hu 等 (2024)。ClusterFusion 则把协作范围扩大到 cluster 级集合通信原语，使跨 thread block 的数据交换与同步也进入融合搜索空间Luo 等 (2025)。这些工作共同说明：逻辑计算图到 GPU kernel 的映射是一个组合优化问题，收益同时取决于 launch 数量、中间数据物化、片上复用、同步和资源占用。

上述方法主要在代码生成阶段决定“切成几个 kernel、每个装什么”。Megakernel 把问题继续推进到设备端执行：多个算子已经进入同一逻辑任务图之后，依赖由何种运行时机制承载、任务以何种粒度投放、逻辑任务图映射为几个物理 grid，成为新的物理执行计划变量。

2.2 Persistent Megakernel

现有 Megakernel 实现大体沿两条路径展开。FlashFormer 这类 whole-model kernel 以固定 cooperative grid 执行一层或多层 Transformer 阶段：面向低 batch 推理，把整模型前向数据流映射到单个 kernel，并通过跨阶段流水缓解权重搬运与计算之间的空隙Nrusimha 等 (2025)；DeepGEMM 的 MegaMoE 则面向专家路由场景，在 SM90 上把 dispatch、专家侧 GEMM、SwiGLU 与 combine 等阶段组织进同一 cooperative kernel，由 grid 内协作完成跨阶段数据交接与同步推进DeepSeek-AI (2026a,b)。二者工作分工与同步位置多在编译期确定，执行时依靠 grid-wide 协作推进固定阶段序列，launch 与中间物化开销低，但同步粒度相对较粗。

另一条路径采用常驻 workers 执行设备端 task graph。Hazy Research Megakernels 将算子任务编译为各 SM 的指令队列，worker 根据 event、counter 和 barrier 推进跨算子依赖Hazy Research (2025a,b)；MPK 进一步提出 SM-level graph 表示，将张量程序降低为细粒度 tasks，并由去中心化的设备内运行时在 persistent megakernel 中调度，论文报告端到端时延改进最高达到 1.7×Cheng 等 (2026)。与静态 cooperative 方案相比，task-graph runtime 能够在依赖允许时让不同 SM 独立推进，从而把整 grid 同步细化为 task 或 tile 级事件。

上述实现调度结构不同，却承担相近的系统职责：一次物理 grid 需要在多个阶段间维持任务发现、ready 判断、数据可见性和 worker 前向进展。其关键收益来自消除 launch 与中间物化，并尽可能把局部 readiness 转化为跨算子流水；其关键成本来自设备端调度、软件同步和不同阶段共享同一执行状态。

2.3 动态 task/event 与自动执行计划生成

真实推理中的 sequence length、batch size 和 MoE routing 会改变 task 数量及依赖关系。Event Tensor 把事件状态提升为 tensor abstraction，用静态和动态 event update/trigger 表达 shape-dependent 与 data-dependent task graph，并由编译器生成 persistent kernel 的静态或动态调度Jin 等 (2026)。Ada-MK 针对固定部署配置执行细粒度 DAG 离线搜索，把运行时分支前移到编译期，同时用异构引擎分别承载 prefill 与 decodeDong 等 (2026)。两项工作从不同方向说明，Megakernel 执行计划由 workload shape、依赖结构和硬件资源共同确定。

自动化工具也开始进入 Megakernel 代码生成。AutoMegaKernel 以 schedule

IR validator 检查 agent 生成计划的 race、deadlock 和资源约束，并从统一代码生成面向多代 NVIDIA GPU 的 persistent cooperative kernel Jaber 等 (2026)。这一进展使执行计划搜索具备更大的候选空间，也提高了对可验证依赖表示和可复现实验接口的要求。对物理分区而言，边界决策需要与 task schedule 一同进入候选计划，并通过依赖合法性检查后再进行性能选择。

2.4 CUDA Graph 与 Programmatic Dependent Launch

CUDA Graph 把 kernel、memcpy 与同步节点预先实例化为可重复执行的 DAG，从而摊销逐 kernel 的主机提交成本；默认 dependency edge 在上游节点完成后放行下游节点 NVIDIA (2026a)。这一机制降低了多 grid 计划的 host-side cost，并保留每个 grid 独立的资源配置和完成语义，因而常作为物理分区之后的提交与重放载体。

Hopper 及后续架构提供 Programmatic Dependent Launch (PDL)。当 producer 的全部 blocks 分别显式触发 programmatic completion，或在退出时完成隐式触发后，consumer grid 获得提前调度机会；consumer 先执行不读取 producer 输出的独立前缀，并在 dependent read 前调用 cudaGridDependencySynchronize()。驱动对实际并发采用机会式调度，执行计划的正确性由同步语义独立保证 NVIDIA (2026d)。CUDA Graph 也可以用 programmatic edge data 表达同一依赖。

CUTLASS 将 PDL 接入 GEMM launch API，其示例主要展示：在已经分立的 producer-consumer GEMM 上，把 consumer 的独立权重预取放到 dependency synchronization 之前，从而使权重访问与 producer 尾部重叠 NVIDIA (2026b,c)。这一用法证明了 PDL 交接接口的可行性，但决策空间仍较窄——物理边界事先给定，可重叠工作也基本限定为固定模式的权重预取。本课题面向 Megakernel 任务图，把依赖分析、边界选择与 PDL 配置放进同一框架：先判断一条边应保留在同一 grid 内，还是映射为跨 grid 完成语义；再为选定物理边界决定 producer trigger 时机、可提前执行的独立前缀形态，以及预取量与存储层级。

2.5 现状归纳与研究机会

现有研究已经形成三个相互衔接的优化层次。图级融合与编译期 kernel 划分决定逻辑计算被切成哪些 kernel；Megakernel 编译器与运行时选择 task 粒度、

事件表示和设备端调度；CUDA Graph 与 PDL 优化多个物理 grid 的提交和交接。三个层次分别成熟，却尚未被系统地贯通：逻辑上已经融合的任务图，仍然缺少一套依据依赖形态选择物理执行方式的方法；PDL 虽已证明可在既定边界上重叠独立工作，却尚未与“该不该划边界、何时放行后继、以何种机制调度”一并纳入同一决策过程。

由此形成若干明确的研究机会。第一，建立明确的抽象层次并基于此进行依赖分析，明确计算图中每条相邻算子边的处理范式——例如保留同一 grid 内的细粒度事件同步或映射为跨 grid 的完成语义——并在此候选集合上搜索覆盖全图的边处理方案，使局部边的选择能够组成全局更优的执行计划。第二，对已被判定为跨 grid 的边，可进一步以 PDL 为硬件基础，构建依赖感知的跨边界重叠方案：把 Megakernel 侧的就绪与投放约束、与 CUDA Graph / PDL 侧的触发时机和独立前缀衔接起来，使跨 grid 交接不再停留在既定边界上的固定预取示例，而能够依据依赖形态选择细粒度可重叠工作与放行时机。第三，基于上述抽象建立面向任意计算图的完整框架：从任务拆解、依赖刻画到执行计划生成与运行时落地，并在真实解码负载上检验端到端收益。

三 课题主要研究内容、预期目标

3.1 问题定义与总体目标

本课题面向低时延 LLM 推理，旨在构建**依赖感知的 Megakernel 推理融合框架与系统**。输入是任意计算图；框架首先通过依赖分析与任务拆解，为图中每条相邻算子边选择合适的执行方式，并生成可落地的端到端执行计划。核心问题因此包含前后衔接的两层决策：其一，依据 task / tile 级依赖形态，明确各边融合方案并在全图上搜索最优边处理方案；其二，对跨 grid 边进一步以 PDL 组织依赖感知的跨边界重叠。

形式化地，设输入计算图为 $G = (O, E)$ ，其中 O 为算子集合， E 为算子间数据依赖。框架将每个算子 $o \in O$ 拆解为可调度的 task 集合 $\mathcal{T}(o)$ ；对相邻算子边 $e = (o_p, o_c)$ ，进一步得到从生产者 task 到消费者 task 的依赖关系。该关系刻画消费者需要等待哪些生产者 task、依赖按 task/tile 粒度局部就绪还是全量就绪，以及消费者在读取依赖数据前是否存在可独立执行的工作。

对每条边 e ，框架生成调度候选集合 $\mathcal{M}(e)$ ，并将边处理方案表示为 $\pi(e) = \langle m_e, \theta_e \rangle$ ，其中 $m_e \in \mathcal{M}(e)$ 为处理范式， θ_e 为相应的同步、投放或 PDL 参数。本课题重点研究表 ?? 中四类候选。前两类将边保留在同一 grid 内，分别利用局部就绪或全量就绪推进消费者；后两类将边映射为物理 grid 边界，并进一步决定是否使用 PDL。全图执行计划记为 $\Pi = \{\pi(e) \mid e \in E\}$ 。边的选择并非彼此独立：一条边是否跨 grid 会改变相邻算子的 grid 分组、资源状态和其它边的可重叠窗口，因此需要在全图上联合求解。

基于以上统一依赖模型，总体优化目标为

$$\Pi^*(G, x) = \arg \min_{\Pi \in \mathcal{F}(G, x)} T_{\text{decode}}(G, x; \Pi), \quad (1)$$

其中 x 表示输入规模与运行配置， $\mathcal{F}(G, x)$ 表示满足数据依赖、内存可见性和硬件资源约束的合法计划集合， T_{decode} 表示完整解码路径的端到端时延。式 (1) 将每条边的处理范式及参数作为全图决策变量，联合确定 grid 分组、同步方式与 PDL 配置。

3.2 主要研究内容

围绕上述目标，本课题重点研究以下四个问题。

1. **计算图依赖关系建模**。建立从算子图到 task graph 的 lowering 过程，以统一抽象刻画相邻算子边的 task/tile 级依赖关系，并据此构造各边合法的处理候选空间。
2. **依赖感知的 PDL 跨边界重叠**。在依赖抽象之上，研究跨 grid 边的 PDL 变换：依据就绪关系识别可提前发起的重叠工作，在保持同步合法的前提下，确定重叠范围、发起时机与执行形态，使生产者与消费者之间的可重叠部分尽可能隐藏边界等待并优化数据读取。
3. **全图边处理方案搜索**。面向全图构造边处理与 grid 分组的联合搜索问题；以静态分析、轻量运行时特征与性能模型估计各候选的收益与开销，在合法空间内求解式 (1)，得到端到端时延最优的执行计划。
4. **框架实现与端到端验证**。实现计划生成、CUDA 代码生成与运行时执行，将所得方案接入 decoder layer、完整 decode step 和连续 token 生成；真实的推理 workload 中进行精度验证的和性能测试。

3.3 预期目标、创新点与成果

3.3.1 预期目标

与主要研究内容一一对应，本课题预期达到：

1. 完成计算图依赖关系建模，形成从算子图到 task graph 的统一抽象，并为相邻算子边给出合法的处理候选空间。
2. 完成依赖感知的 PDL 跨边界重叠方法，能够依据就绪关系识别重叠工作，并在同步合法约束下确定跨 grid 边的重叠方案。
3. 完成全图边处理与 grid 分组的联合搜索，在合法空间内求解式 (1)，得到端到端时延更优的执行计划。
4. 完成框架原型与端到端验证，贯通计划生成、CUDA 代码生成与运行时执行，并在真实推理 workload 上完成精度与性能评估。

3.3.2 预期创新点

1. 提出面向任意计算图的依赖感知边处理抽象，以 task/tile 级依赖关系系统一刻画同 grid 与跨 grid 候选，形成可搜索的合法方案空间。
2. 提出依赖感知的 PDL 跨边界重叠方法，将重叠工作识别、发起时机与执行形态纳入统一变换，而不仅是固定边界上的静态预取。
3. 提出覆盖计算图全部边的联合执行计划搜索方法，使边处理选择与 grid 分组协同优化，直接服务于端到端 decode 时延目标。

3.3.3 预期成果

预期形成依赖感知的 Megakernel 推理融合框架与系统原型，覆盖依赖建模、PDL 跨边界重叠、全图计划搜索及代码生成运行时；并给出真实推理 workload 上的精度与性能实验结果，以及硕士学位论文和相关学术论文。

四 拟采用的研究方法、技术路线与实验方案

4.1 框架总体方法

本课题以计算图为输入，按“依赖关系建模—PDL 跨边界重叠—全图边处理搜索—计划生成与运行时执行”组织技术路线，整体关系如图 3 所示。Megakernel 承载同 grid 融合与设备端调度；当全图方案在某条边上选择物理分区时，再生成多个 grid 及其 CUDA Graph / PDL 依赖。

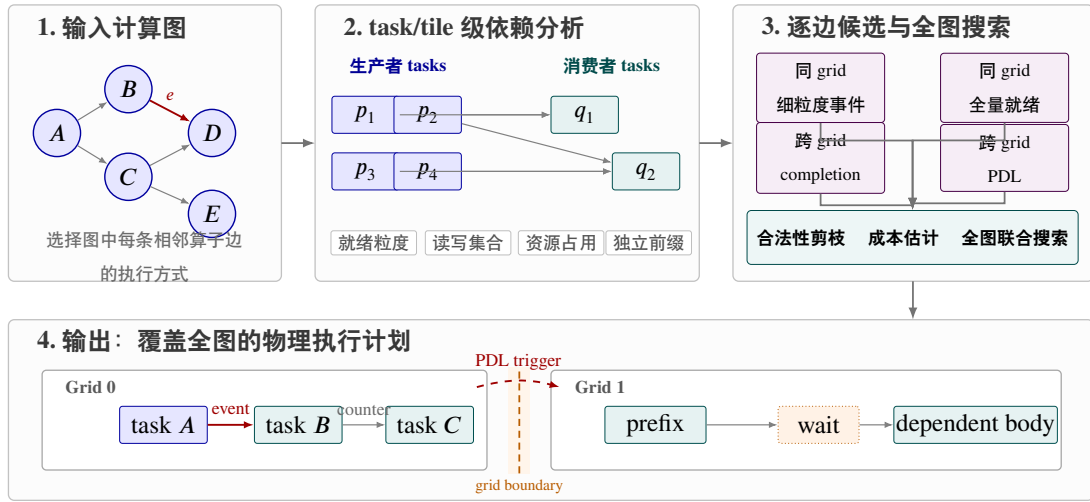


图 3 依赖感知的计算图边调度框架。框架从任意计算图出发，在 task/tile 粒度分析每条边的就绪关系、读写集合、资源占用与独立前缀；随后为各边生成候选并进行全图联合搜索，最终输出 grid 分组、边同步方式和 PDL 配置。

框架保留两层表示。算子图层记录算子、张量及相邻算子边，用于决定融合区域与物理 grid 分组；task graph 层记录算子拆解后的 task/tile 及其依赖，用于判断消费者何时可以开始。两层保持映射，使 task 级依赖分析能够指导算子边的处理方式，而最终方案仍可还原为完整计算图上的物理执行计划。

4.2 计算图依赖关系建模

对算子 o ，框架根据并行维度与实现模板生成 task 集合 $\mathcal{T}(o)$ 。对相邻算子边 $e = (o_p, o_c)$ 及任一消费者 task $c \in \mathcal{T}(o_c)$ ，定义其直接依赖集合

$$\text{Dep}_e(c) = \{p \in \mathcal{T}(o_p) \mid c \text{ 读取 } p \text{ 产生的数据}\}.$$

该表示构成边级依赖抽象的核心：据此区分局部就绪与全量就绪，并构造各边合法的处理候选空间，覆盖同 grid 与跨 grid 两类执行形态。候选生成只给出合法

空间，最终选择由全图搜索完成。

4.3 依赖感知的 PDL 跨边界重叠

当方案在边 $e = (o_p, o_c)$ 上选择跨 grid 执行时，框架将消费者工作划分为独立前缀 C_e^{ind} 与依赖主体 C_e^{dep} 。独立前缀必须满足

$$\begin{aligned} R(C_e^{\text{ind}}) \cap W(\mathcal{T}(o_p)) &= \emptyset, \\ W(C_e^{\text{ind}}) \cap (R(\mathcal{T}(o_p)) \cup W(\mathcal{T}(o_p))) &= \emptyset, \end{aligned} \quad (2)$$

其中读写集合包含别名传递关系。消费者的 dependency wait 必须支配 C_e^{dep} 中每一次对生产者输出的读取，从而保证提前执行不改变数值语义。

在此约束下，框架依据就绪关系识别可提前发起的重叠工作，并确定重叠范围、发起时机与执行形态，使生产者与消费者之间的可重叠部分尽可能隐藏边界等待。与仅在固定边界上移动权重预取不同，本课题从 task graph 与消费者实现中共同提取可提前工作，并将 trigger、wait 与重叠阶段参数纳入统一变换。

4.4 全图边处理方案搜索

面向全图构造边处理与 grid 分组的联合搜索问题。框架以静态分析、轻量运行时特征与性能模型估计各候选的收益与开销；给定方案 Π 后，沿 task graph 与物理 grid 顺序传播时间，得到端到端估计 $\hat{T}(G, x; \Pi)$ 。搜索按拓扑顺序展开，状态记录已形成的 grid 分组、关键路径与资源包络，在合法空间内求解式 (1)。对规模较小的子图可辅以穷举校验；最终少量候选在目标 GPU 上实测后部署。搜索按 workload bucket 进行，运行时按当前输入选择已生成计划，不在每个 token 上重新搜索。

4.5 计划生成与运行时执行

全图搜索结束后，框架将由同 grid 边连接的 task 组织为物理 grid，并生成相应同步与调度结构；跨 grid 边生成独立 kernel 节点及完成依赖。多个 grid 由 CUDA Graph 统一提交。对选择 PDL 的边，代码生成器插入 programmatic completion trigger、dependent launch 属性，以及首次依赖读取前的 grid dependency wait。运行时按计划重放执行，并在中间张量与最终输出上检查正确性，确保不同物理映射保持相同计算语义。

4.6 实验方案

实验按主要研究内容组织，并对应上述基本实现路径。

1. **依赖关系建模**。实现算子图到 task graph 的 lowering、边依赖分析与合法候选生成；在真实 LLM 计算图上检查依赖还原完整性与候选空间可枚举性。
2. **PDL 跨边界重叠**。实现依赖感知的 PDL 变换，并对照普通 grid completion 与启用重叠的方案，报告可隐藏工作量、实际重叠窗口及端到端时延变化。
3. **全图边处理搜索**。实现联合搜索原型；对照固定单 grid、固定多 grid、逐边贪心与全图搜索，比较关键路径、搜索开销与 decode 时延。
4. **框架实现与端到端验证**。贯通计划生成、代码生成与运行时执行，接入 decoder layer、完整 decode step 与连续 token 生成；在真实推理 workload 上完成精度验证与性能测试。主要指标包括正确性、decode latency、TPOT、P50/P90/P99，以及搜索与计划生成开销。

五 已有科研基础与所需的科研条件

5.1 已有科研基础

本课题已有工作提出了 DACOS (Dependency-Aware Cross-Kernel Overlapping)，即一种依赖感知的跨 kernel 重叠框架[Hao 等 \(2026\)](#)。DACOS 利用 programmatic dependent launch 提前发起后继 kernel，并在前驱输出尚未就绪前执行与依赖无关的工作，例如操作数准备与数据预取。框架结合依赖分析、跨 kernel 重叠构造以及代价模型驱动的配置选择，共同决定后继 kernel 的触发时机与可提前推进的工作内容。在真实 LLM workload 上的评估表明，相对 TorchEager 与 TorchInductor，DACOS 分别可获得最高约 3.5× 与 1.2× 的端到端加速，验证了依赖感知跨 kernel 重叠的有效性。

上述基础为本课题提供了可复用的依赖分析、重叠阶段构造与 PDL 参数化实现路径。本课题进一步面向任意计算图，将边级依赖抽象、PDL 跨边界重叠与全图边处理搜索统一到同一融合框架中。

5.2 所需的科研条件

实验环境具备 NVIDIA Hopper 级 GPU、CUDA 工具链及完整 CUDA/C++ 构建条件，可支持依赖分析、计划搜索、代码生成与端到端验证。已有代码、配置与评测脚本可作为后续原型扩展的起点。

六 研究工作计划与进度安排

研究工作按主要研究内容推进：依赖关系建模、PDL 跨边界重叠、全图边处理搜索，以及框架实现与端到端验证。阶段安排与可验收成果如下。

表 1 研究进度安排与阶段成果

时间	主要工作	阶段成果
2026.07–2026.08	完成算子图到 task graph 的 lowering；实现相邻算子边的依赖抽象与合法候选空间；在真实计算图上校验依赖还原	依赖分析 IR、候选生成接口与依赖报告
2026.09–2026.10	实现依赖感知的 PDL 跨边界变换；识别重叠工作并确定发起时机与执行形态；相对 grid completion 完成边级重叠验证	PDL 变换原型、合法性检查与边级重叠结果
2026.11–2026.12	实现边处理与 grid 分组的联合搜索及成本估计；在 decoder layer 上比较固定方案、逐边贪心与全图搜索	全图计划搜索器及 decoder-layer 时延结果
2027.01–2027.02	贯通计划生成、CUDA 代码生成与运行时执行；接入完整 decode step 与连续 token 生成，完成正确性通路	可运行框架原型与端到端正确性结果
2027.03–2027.04	在真实推理 workload 上完成精度与性能测试；扫描 batch、context 与模型 shape，并做依赖/PDL/搜索消融	跨图时延、TPOT/P90/P99、搜索开销与消融结果
2027.05–2027.06	完善系统原型与复现材料，完成论文撰写	依赖感知融合框架、可复现实验包、投稿稿件和学位论文

参考文献

- Agrawal A, Kedia N, Panwar A, et al. Taming throughput-latency tradeoff in LLM inference with sarathi-serve [C/OL]//18th USENIX Symposium on Operating Systems Design and Implementation. 2024: 117-134. <https://www.usenix.org/conference/osdi24/presentation/agrawal>.
- Chen T, Moreau T, Jiang Z, et al. TVM: An automated end-to-end optimizing compiler for deep learning [C/OL]//13th USENIX Symposium on Operating Systems Design and Implementation. 2018: 578-594. <https://www.usenix.org/conference/osdi18/presentation/chen>.
- Cheng X, Zhang Z, Zhou Y, et al. MPK: A compiler and runtime for mega-kernelizing tensor programs [C/OL]//20th USENIX Symposium on Operating Systems Design and Implementation (OSDI 26). Seattle, WA: USENIX Association, 2026: 1909-1926. <https://www.usenix.org/conference/osdi26/presentation/cheng>.
- Dao T. Flashattention-2: Faster attention with better parallelism and work partitioning [C]//The Twelfth International Conference on Learning Representations. 2024.
- Dao T, Fu D Y, Ermon S, et al. Flashattention: Fast and memory-efficient exact attention with IO-awareness [C]//Koyejo S, Mohamed S, Agarwal A, et al. Advances in Neural Information Processing Systems: volume 35. Curran Associates, Inc., 2022: 16344-16359.
- DeepSeek-AI. DeepGEMM: Clean and efficient FP8 GEMM kernels [EB/OL]. 2026[2026-07-20]. <https://github.com/deepseek-ai/DeepGEMM/tree/559d79fb6994a58b8a15b4b93bf13ccc16edf247>.
- DeepSeek-AI. SM90 cooperative MegaMoE implementation in DeepGEMM [EB/OL]. 2026[2026-07-20]. <https://github.com/deepseek-ai/DeepGEMM/commit/f98330768857d65025ca457bf2e9ca168f5fd2c5>.
- Dong W, et al. Ada-mk: Adaptive megakernel optimization via automated DAG-based search for LLM inference [EB/OL]. 2026. <https://arxiv.org/abs/2605.11581>.
- Hao Z, Li G, Luo F, et al. DACOS: Dependency-aware cross-kernel overlapping for optimizing short-sequence workloads in LLM applications [Z]. 2026.
- Hazy Research. Look ma, no bubbles! designing a low-latency megakernel for LLM inference [EB/OL]. 2025[2026-06-30]. <https://hazyresearch.stanford.edu/blog/2025-05-27-no-bubbles>.
- Hazy Research. Megakernels: Low-latency LLM inference with device-side task graphs [EB/OL]. 2025[2026-07-20]. <https://github.com/HazyResearch/Megakernels/tree/7309cec801537b61fea3b50d7dfe454a6cde578e>.
- Hu M, Venkatram A, Biswas S, et al. Optimal kernel orchestration for tensor programs with Korch [C/OL]//Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3. ACM, 2024: 755-769. <https://doi.org/10.1145/3620666.3651383>.

- Jaber J, Jaber O. AutoMegaKernel: A statically-checked agent harness for self-retargeting megakernel synthesis [EB/OL]. 2026. <https://arxiv.org/abs/2606.09682>. DOI: 10.48550/arXiv.2606.09682.
- Jin H, Hou B, Wang G, et al. Event tensor: A unified abstraction for compiling dynamic megakernel [C/OL]//Proceedings of Machine Learning and Systems: volume 8. Bellevue, WA, USA, 2026 [2026-07-20]. <https://openreview.net/forum?id=PJqFhAbUHa>.
- Kwon W, Li Z, Zhuang S, et al. Efficient memory management for large language model serving with PagedAttention [C/OL]//Proceedings of the 29th Symposium on Operating Systems Principles. 2023: 611-626. DOI: 10.1145/3600006.3613165.
- Luo F, et al. Agentix: An efficient serving engine for LLM agents as general programs [EB/OL]. 2026. <https://www.usenix.org/conference/nsdi26/presentation/luo>.
- Luo X, Liu Z, Zhou Y, et al. Clusterfusion: Expanding operator fusion scope for LLM inference via cluster-level collective primitive [C]//Belgrave D, Zhang C, Lin H, et al. Advances in Neural Information Processing Systems: volume 38. Curran Associates, Inc., 2025.
- Niu W, Guan J, Wang Y, et al. DNNFusion: Accelerating deep neural networks execution with advanced operator fusion [C/OL]//Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation. 2021: 883-898. DOI: 10.1145/3453483.3454083.
- Nrusimha A, Brandon W, Mishra M, et al. Flashformer: Whole-model kernels for efficient low-batch inference [EB/OL]. 2025. <https://arxiv.org/abs/2505.22758>.
- NVIDIA. CUDA programming guide 13.3: CUDA graphs [EB/OL]. 2026[2026-07-20]. <https://docs.nvidia.com/cuda/cuda-programming-guide/04-special-topics/cuda-graphs.html>.
- NVIDIA. CUTLASS 4.5.1 documentation: Dependent kernel launches [EB/OL]. 2026[2026-07-20]. https://docs.nvidia.com/cutlass/4.5.1/media/docs/cpp/dependent_kernel_launch.html.
- NVIDIA. CUTLASS example 63: Hopper GEMM with L2 weight prefetch [EB/OL]. 2026[2026-07-20]. <https://github.com/NVIDIA/cutlass>.
- NVIDIA. CUDA programming guide 13.3: Programmatic dependent launch and synchronization [EB/OL]. 2026[2026-07-20]. <https://docs.nvidia.com/cuda/cuda-programming-guide/>.
- Shah J, Bikshandi G, Zhang Y, et al. Flashattention-3: Fast and accurate attention with asynchrony and low-precision [C/OL]//Globerson A, Mackey L, Belgrave D, et al. Advances in Neural Information Processing Systems: volume 37. Curran Associates, Inc., 2024: 68658-68685. DOI: 10.52202/079017-2193.
- Shi Y, Yang Z, Xue J, et al. Welder: Scheduling deep learning memory access via tile-graph [C/OL]//17th USENIX Symposium on Operating Systems Design and Implementation. 2023: 701-718. <https://www.usenix.org/conference/osdi23/presentation/shi>.

- Vellaisamy P, Tripathi S, Natarajan V, et al. Taxbreak: Unmasking the hidden costs of LLM inference through overhead decomposition [EB/OL]. 2026. <https://arxiv.org/abs/2603.12465>.
- Yu G I, Jeong J S, Kim G W, et al. Orca: A distributed serving system for transformer-based generative models [C/OL]//16th USENIX Symposium on Operating Systems Design and Implementation. 2022: 521-538. <https://www.usenix.org/conference/osdi22/presentation/yu>.
- Zhong Y, Liu S, Chen J, et al. Distserve: Disaggregating prefill and decoding for goodput-optimized large language model serving [C/OL]//18th USENIX Symposium on Operating Systems Design and Implementation. 2024: 193-210. <https://www.usenix.org/conference/osdi24/presentation/zhong-yinmin>.